Studio Nx

Backend(FastAPI/Python) 95 — 묶음 C 탈퇴한 계정

2026-10-12

Backend(FastAPI/Python) 95 — 묶음 C: 탈퇴한 계정

항목	내용
파일	`studionx/backend/tests/test_phase1_acceptance.py` (묶음 C, 시험 8개)
작업	P1-18 완료 기준 시험: PRD 01 §10의 7번 "탈퇴한 계정은 로그인되지 않는다" (FR-108 재가입 규칙 포함)
작성	Claude (AI), 2026-10-13, 사용자 검토 (묶음 단위)
이전 · 다음	94 묶음 B · (P1-18 끝, 개요는 61번)

1. 하는 일

가입, 로그인, 프로젝트 하나를 만든 회원이 **실제 요청으로 탈퇴**하는 상황을 만들고, 그 뒤의 모든 문이 닫혔는지와 3일 재가입 규칙이 지켜지는지를 확인합니다. 시험 시작은 `withdrawn` 도우미가 맡습니다 (회원 번호, 탈퇴 전에 받은 헤더, 프로젝트 번호를 돌려줍니다).

시험	확인하는 것
`test_c1`	탈퇴 전에 받아 둔 Access Token으로 `/users/me`, `/me/dashboard`, `/projects`, `/projects/{id}`, 프로젝트 생성, 정보 수정을 모두 시도하면 401입니다. 재발급도 401입니다
`test_c2`	탈퇴한 계정으로 로그인하면 **없는 이메일과 응답이 완전히 같습니다**(401 `INVALID_CREDENTIALS`). 쿠키는 받지 않습니다
`test_c3`	비밀번호가 틀리면 400 `WRONG_PASSWORD`이고 아무것도 바뀌지 않습니다. 계속 로그인 상태이고 다시 로그인도 됩니다
`test_c4`	탈퇴 직후에도 이메일과 닉네임은 바뀌지 않고, `status`는 `DELETED`, `deleted_at`이 채워집니다
`test_c5`	3일 안에는 중복 확인 API가 `available: false`, `reason: REJOIN_TOO_SOON`, 가능해지는 시각을 줍니다. 같은 이메일로 가입하면 409 `REJOIN_TOO_SOON`(세부 정보에 시각 포함), 같은 닉네임으로 가입해도 409 `REJOIN_TOO_SOON`입니다
`test_c6`	3일 경계가 정확합니다. 탈퇴한 지 2일 23시간 59분이면 409, 3일 1분이면 가입됩니다 (201)
`test_c7`	3일 뒤에는 같은 이메일과 닉네임으로 다시 가입됩니다. 새 계정은 새 번호이고, 예전 행은 `DELETED`로 남고 이메일은 `deleted+번호@withdrawn.invalid`, 닉네임은 `deleted_user_…`로 비워집니다. 새 계정은 예전 프로젝트를 물려받지 않고(목록 비어 있음, 예전 프로젝트 404), 예전 비밀번호로는 새 계정에 못 들어갑니다. 중복 확인(GET)은 DB를 바꾸지 않습니다
`test_c8`	탈퇴한 소유자의 프로젝트는 다른 멤버(편집자)가 계속 읽고 고칠 수 있습니다. 응답에 탈퇴한 소유자의 이메일, 닉네임이 보이지 않습니다

2. 코드 (뼈대 부분)

```
@pytest.fixture
def withdrawn(client, db):
    headers = member_headers(client, "acc_gone")
    user_id = client.get(f"{API}/users/me", headers=headers).json()["user_id"]
    pid = client.post(PROJECTS, json={"title": "Left Behind"}, headers=headers).json()["project_id"]
    assert withdraw(client, headers).status_code == 204
    return user_id, headers, pid


def set_deleted_ago(db, user_id, **ago):
```

```
    user = db.get(User, user_id)
    user.deleted_at = datetime.now(UTC).replace(tzinfo=None) - timedelta(**ago)
    db.flush()
    db.expire_all()
```

식	뜻
`withdraw(client, headers, password)`	`DELETE /users/me`에 비밀번호를 본문으로 보냅니다 (`client.request`로 `DELETE`에 본문을 실었습니다)
`set_deleted_ago(db, id, days=3, minutes=1)`	탈퇴 시각을 과거로 돌려서 "3일이 지난" 상황을 만듭니다. 시험에서 실제로 3일을 기다릴 수 없기 때문입니다
`db.expire_all()`	세션이 기억한 값을 버려서, 서버가 바꾼 값을 다시 읽게 합니다
`timedelta(**ago)`	`days=2, hours=23, minutes=59`처럼 넘긴 값을 기간으로 만듭니다

3. 왜 이렇게

선택	이유
탈퇴 **전에** 받은 Access Token을 씀	Access Token은 만료(30분)까지 서명이 유효합니다. 서버가 매 요청마다 계정 상태를 확인해야 막히므로, 이 시험이 그 확인이 살아 있는지 봅니다
로그인 응답을 없는 이메일과 비교	"탈퇴한 계정이다"를 알려 주면 누가 탈퇴했는지 알아낼 수 있습니다 (FR-103, 108)
3일 경계를 1분 단위로	경계 오류(`<`와 `<=`, 시간대 혼동)는 정확히 경계에서만 드러납니다
중복 확인 API는 DB를 바꾸지 않음을 확인	읽기 요청이 계정 이름을 비우면 누구나 남의 탈퇴 계정을 풀 수 있게 됩니다. 가입할 때만 비웁니다
새 계정이 프로젝트를 물려받지 않는지	이메일이 같아도 번호가 다른 별개의 회원입니다
탈퇴한 소유자의 프로젝트를 남김	사용자 결정(2026-10-03)입니다. 다른 멤버의 작업이 사라지지 않습니다

4. 시험이 의미 있는지 확인 (일부러 망가뜨리기)

확인한 뒤 모두 되돌렸고, `app/` 아래에는 바뀐 파일이 없습니다.

바꾼 코드	실패한 시험
재가입 대기 `REJOIN_WAIT_DAYS`를 3일에서 1일로	C6 (경계)
토큰 검사에서 "정상 계정만 허용"을 끔 (`deps.py`)	C1 (탈퇴 뒤 잠금)

5. 결과

8개 모두 통과했습니다. 코드 수정은 없었고, 기존 코드가 탈퇴 관련 완료 기준을 이미 만족한다는 확인입니다. 정책이 불분명해서 질문한 곳은 없었습니다.